

Exercise 02.01.1 — Cross-Cutting Concerns in Practice

RocketTickets 🎸 — Flash Sale Architecture

Scenario Overview

RocketTickets is a concert ticketing platform. You've just landed the contract to handle the **flash sale** for the most anticipated tour of the year. 100,000 tickets. On sale in 3 days. The previous platform crashed under load and made headlines for the wrong reasons.

Your team has **40 minutes** to design the cross-cutting concern strategy before development starts.

The System

Backend Services

Service	Responsibility
Inventory Service	Tracks available seats per section
Checkout Service	Reserves seats and initiates payment
Payment Service	Processes transactions via Stripe
User Service	Manages accounts, authentication, purchase history
Notification Service	Sends confirmations via email, SMS and push

Frontend Surfaces

Surface	Responsibility
Sale Page	Real-time seat map, countdown timer
Checkout Flow	Seat selection, payment, confirmation
My Tickets	Purchase history and downloadable tickets

The Constraints

Read these carefully. Some have only one right answer. Others have multiple valid approaches — **your job is to argue your choice**.

1. Logging

- All purchase attempts must be logged (successful and failed)
- The payment provider returns full card details in their API response — your logs are stored in plain text on S3
- During the flash sale, 50,000 requests per minute are expected
- GDPR applies — users are EU-based

2. Error Handling

- The payment provider times out occasionally under load (average: 2% of requests during peak)
- A timed-out payment request may or may not have been processed on Stripe's side — you don't know
- The seat map must remain responsive even if the Inventory Service is slow
- Users cannot receive a generic "something went wrong" message — legal requires a reference number for every failed transaction

3. Caching

- The seat map needs to update in near real-time — seeing a seat as available and then being told it's gone is the #1 complaint from the previous platform
- User authentication tokens are validated on every request
- The concert venue layout (sections, rows, seat numbers) never changes
- Purchase history must always reflect the actual database state

4. Notifications

- Users must receive a confirmation within 30 seconds of purchase

- SMS costs €0.05 per message, email costs €0.001
 - The notification must not block the checkout completing
 - If a notification fails to send, the ticket is still valid — but the user doesn't know they have it
-

The Traps

Hidden in the constraints above are decisions that **look obvious but aren't**. Your design should address all of the following — and for each one, explain why the obvious answer is wrong or incomplete:

Trap A — *"Add logging middleware at the API gateway so everything gets logged automatically"*

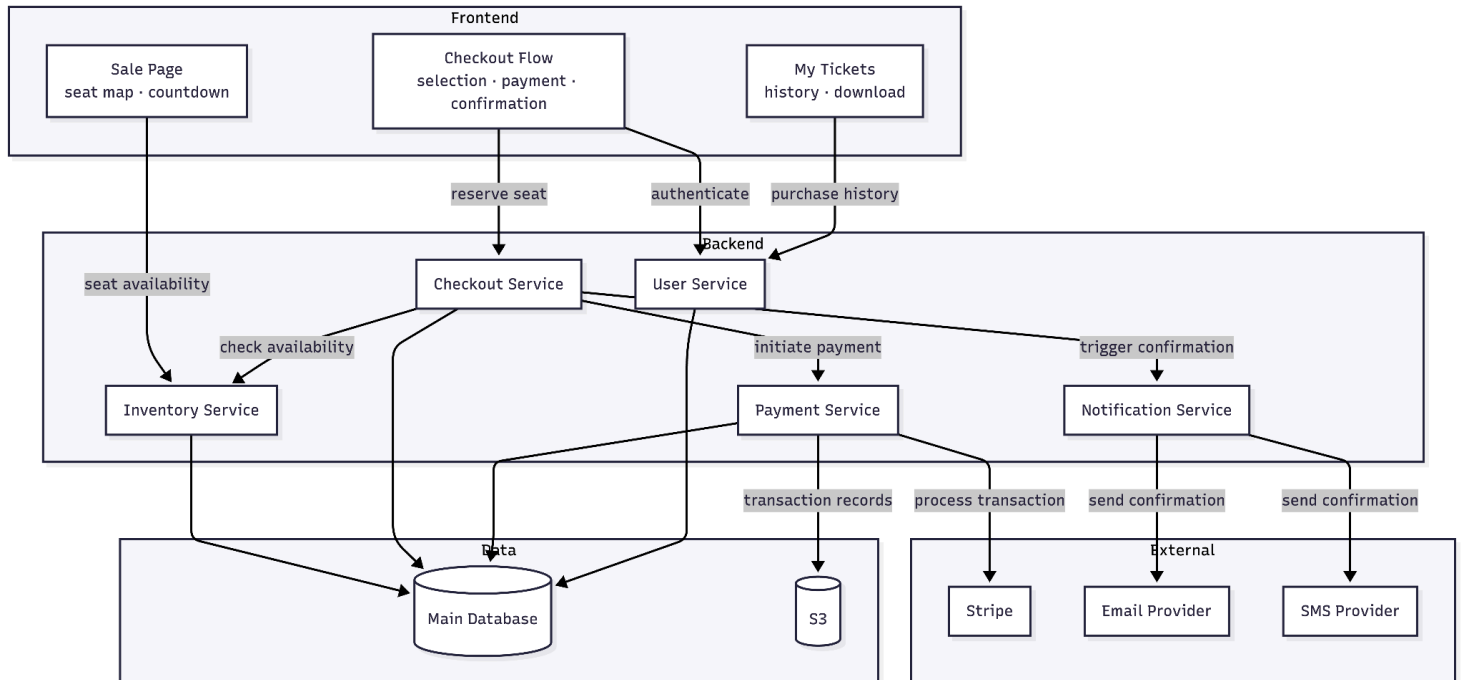
Trap B — *"Use AOP on every method in the Payment Service so we have full observability"*

Trap C — *"Inject a different logger implementation for the Payment Service since its logs need encryption"*

Trap D — *"Use the same error handling middleware for all services"*

Your Task

1. Design the cross-cutting concern strategy for each of the four constraint areas
 2. For each area, identify **which pattern** you are using (Middleware, AOP, Decorator, DI, message queue) and why
 3. Produce a **diagram** showing where each concern lives in the architecture
 4. For **at least two decisions**, present the alternatives you considered and the explicit tradeoff that made you choose one over the other
-



Deliverables

Architecture Diagram

Any format — boxes, flowchart, sequence diagram. Must show where each cross-cutting concern is handled relative to the services.

Decision Table

Concern	Pattern Chosen	Why	What You Rejected	Why You Rejected It
Logging				
Error Handling				
Caching				
Notifications				